Solution: Matrix Diagonal Sum (easy)

Problem Statement

Given a square matrix (2D array), calculate the sum of its two diagonals.

The two diagonals in consideration are the primary diagonal that spans from the top-left to the bottom-right and the secondary diagonal that spans from top-right to bottom-left. If a number is part of both diagonals (which occurs only for odd-sized matrices), it should be counted only once in the sum.

Examples

1. **Example 1**:
   - **Input**:
     ```
     [[1,2,3],
      [4,5,6],
      [7,8,9]]
     ```
   - **Expected Output**: 25
   - **Justification**: Summing up the two diagonals (1+5+9+3+7), we get 25. Please note that the element at [1][1] = 5 is counted only once.
2. **Example 2**:
   - **Input**:
     ```
     [[1,0],
      [0,1]]
     ```
   - **Expected Output**: 2
   - **Justification**: The sum of the two diagonals is 1+1 = 2.
3. **Example 3**:
   - **Input**:
     ```
     [[5]]
     ```
   - **Expected Output**: 5
   - **Justification**: Since there's only one element, it is the sum itself.

Solution

The primary algorithmic approach for this problem is straightforward yet efficient, involving a single loop traversal through the matrix, accumulating the sum of the diagonal elements as it progresses. This approach is effective in that it avoids unnecessary calculations or traversals through the matrix, adhering strictly to the diagonal indices to retrieve the values to be summed. The main mechanism involves identifying and summing the elements of the primary and secondary diagonal, ensuring that, if the matrix size is odd, the central element (which belongs to both diagonals) is not double-counted.

Here's the algorithm for calculating the diagonal sum of a matrix, broken down into steps:

1. **Initialize Sum:** Start by initializing a variable to store the sum of the diagonal elements. Let's call this variable `diagonalSum`.
2. **Loop Through Matrix:** Iterate through the matrix using a loop. Since the matrix is square, you can use a single index to traverse both rows and columns. Let's use `i` as the loop variable, ranging from 0 to the length of the matrix minus one.
3. **Add Primary Diagonal Elements:** In each iteration, add the element at the primary diagonal to `diagonalSum`. The primary diagonal elements are those where the row and column indices are equal, i.e., `matrix[i][i]`.
4. **Add Secondary Diagonal Elements:** In the same iteration, add the element at the secondary diagonal to `diagonalSum`. The secondary diagonal elements are those where the column index is the complement of the row index, i.e., `matrix[i][matrix.length - 1 - i]`.
5. **Avoid Double Counting:** If the matrix has an odd number of rows and columns, the central element will be counted twice (once for each diagonal). To correct this, subtract the central element from `diagonalSum`. The central element is at the position `matrix[middle][middle]`, where `middle = matrix.length / 2`.
6. **Return the Result:** After completing the loop, return the value of `diagonalSum`. This is the sum of the elements on both diagonals of the matrix.

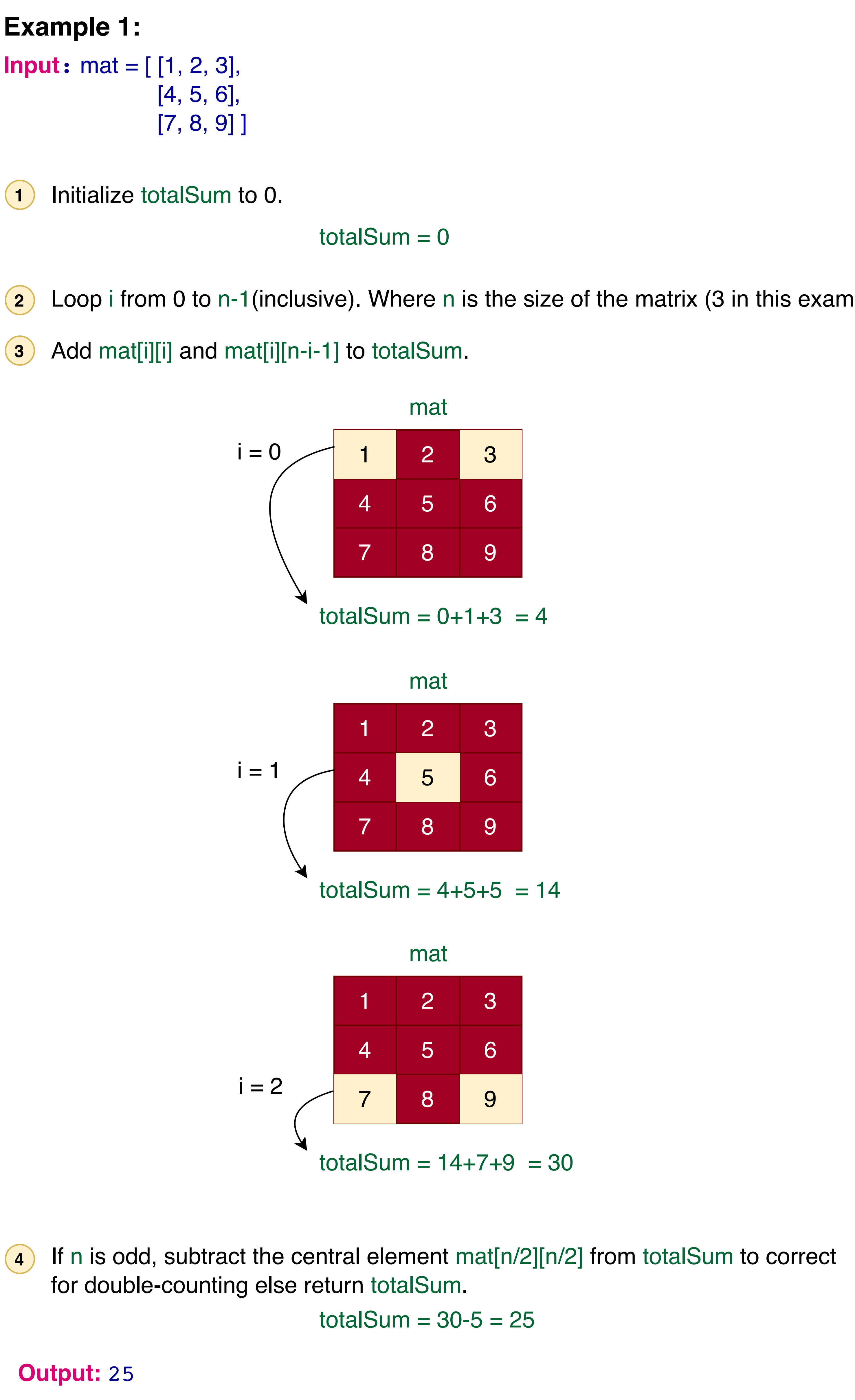
Algorithm Walkthrough

Consider an input matrix:

```
[[1,2,3],
 [4,5,6],
 [7,8,9]]
```

- Initialize `totalSum` to 0.
- Loop `i` from 0 to `n-1` (inclusive). Where `n` is the size of the matrix (3 in this example).
  - Add `mat[i][i]` and `mat[i][n-i-1]` to `totalSum`.
  - For `i=0`, add 1+3 to `totalSum` => `totalSum` = 4.
  - For `i=1`, add 5+5 to `totalSum` => `totalSum` = 14.
  - For `i=2`, add 9+7 to `totalSum` => `totalSum` = 30.
- Since `n` is odd, subtract the central element `mat[n/2][n/2]` (which is 5) from `totalSum` to correct for double-counting => `totalSum` = 25.
- Return `totalSum` which is 25.

Here is the visual representation of the algorithm:



Example 1:

Input: mat = [[1, 2, 3],
[4, 5, 6],
[7, 8, 9]]

① Initialize totalSum to 0.

totalSum = 0

② Loop i from 0 to n-1(inclusive). Where n is the size of the matrix (3 in this example).

③ Add mat[i][i] and mat[i][n-i-1] to totalSum.

totalSum = 0+1+3 = 4

totalSum = 4+5+5 = 14

totalSum = 14+7+9 = 30

④ If n is odd, subtract the central element mat[n/2][n/2] from totalSum to correct for double-counting else return totalSum.

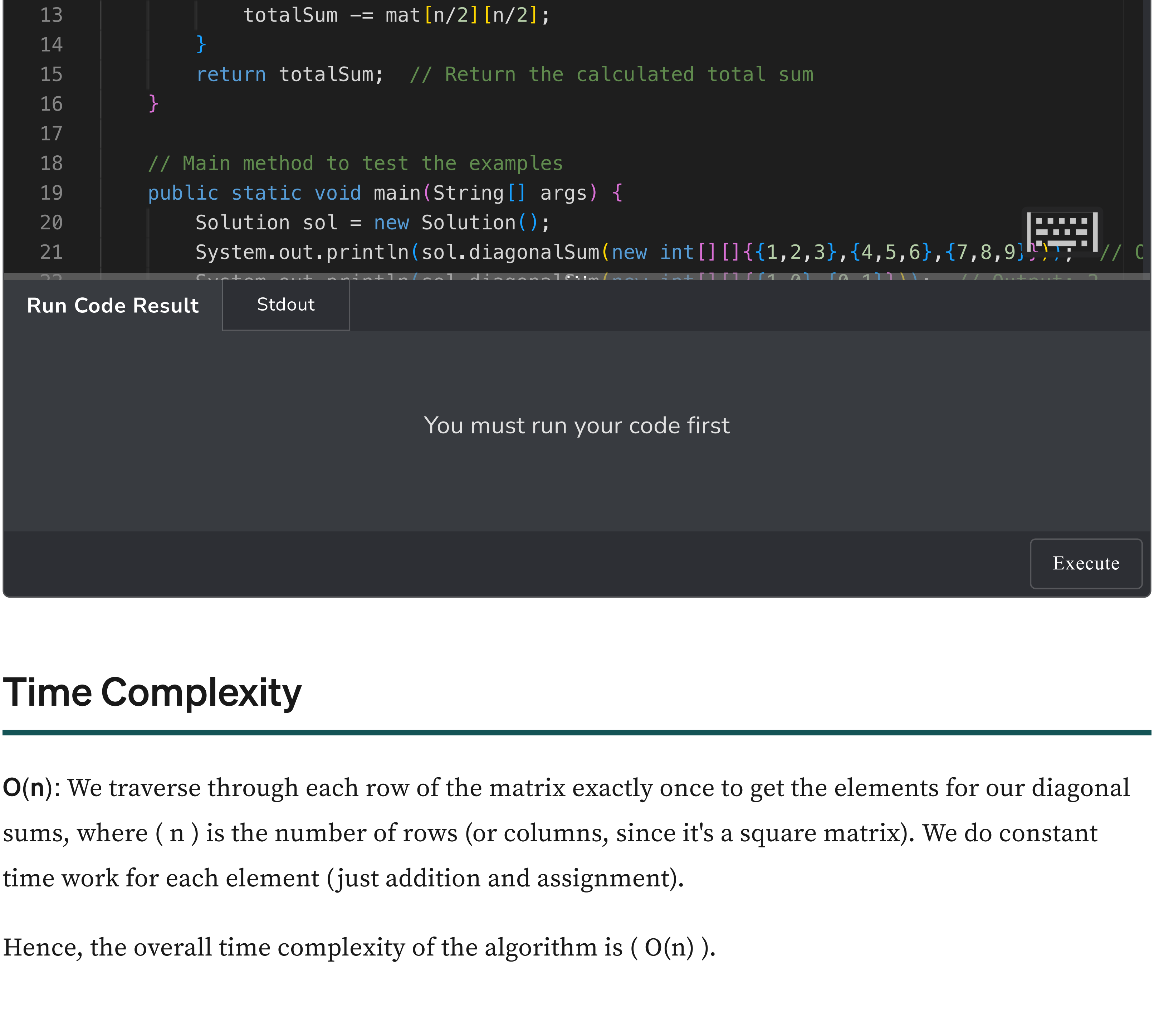
totalSum = 30-5 = 25

Output: 25

Matrix Diagonal Sum

Code

Here is the code for this algorithm:



```java
public class Solution {
    public int diagonalSum(int[][] mat) {
        int n = mat.length;  // Get the size of the matrix
        int totalSum = 0;  // Initialize the total sum

        // Loop through each row
        for (int i = 0; i < n; i++) {
            totalSum += mat[i][i] + mat[i][n-i-1];  // Add primary and secondary diagona
        }

        // If n is odd, subtract the central element
        if (n % 2 != 0) {
            totalSum -= mat[n/2][n/2];
        }
        return totalSum;  // Return the calculated total sum
    }

    // Main method to test the examples
    public static void main(String[] args) {
        Solution sol = new Solution();
        System.out.println(sol.diagonalSum(new int[][]{{1,2,3},{4,5,6},{7,8,9}});  // O
```

You must run your code first

Time Complexity

O(n): We traverse through each row of the matrix exactly once to get the elements for our diagonal sums, where (n) is the number of rows (or columns, since it's a square matrix). We do constant time work for each element (just addition and assignment).

Hence, the overall time complexity of the algorithm is (O(n)).

Space Complexity

O(1): The space complexity is constant because we are using a fixed amount of additional space. Our algorithm only uses two integer variables (`n` and `totalSum`) besides the input, which doesn't grow with the size of the input matrix. Therefore, the amount of extra space utilized doesn't scale with the size of the input.

So, the overall space complexity of the algorithm is (O(1)).

← Previous
Problem 2: Matrix Diagonal Sum (Easy)
Next →
Problem 3: Row With Maximum Ones(Easy)
☐ Mark as Completed